

# Project 05 – Wireshark DNS Analysis: Resolution Flow, Caching & Packet Forensics

**Author:** Oluwaseun Joseph

**Project Type:** Cybersecurity Portfolio – Network & DNS Analysis

**Tools:** Wireshark, nslookup

**Protocols:** DNS (UDP/53), HTTP, HTTPS

## 1. Project Overview

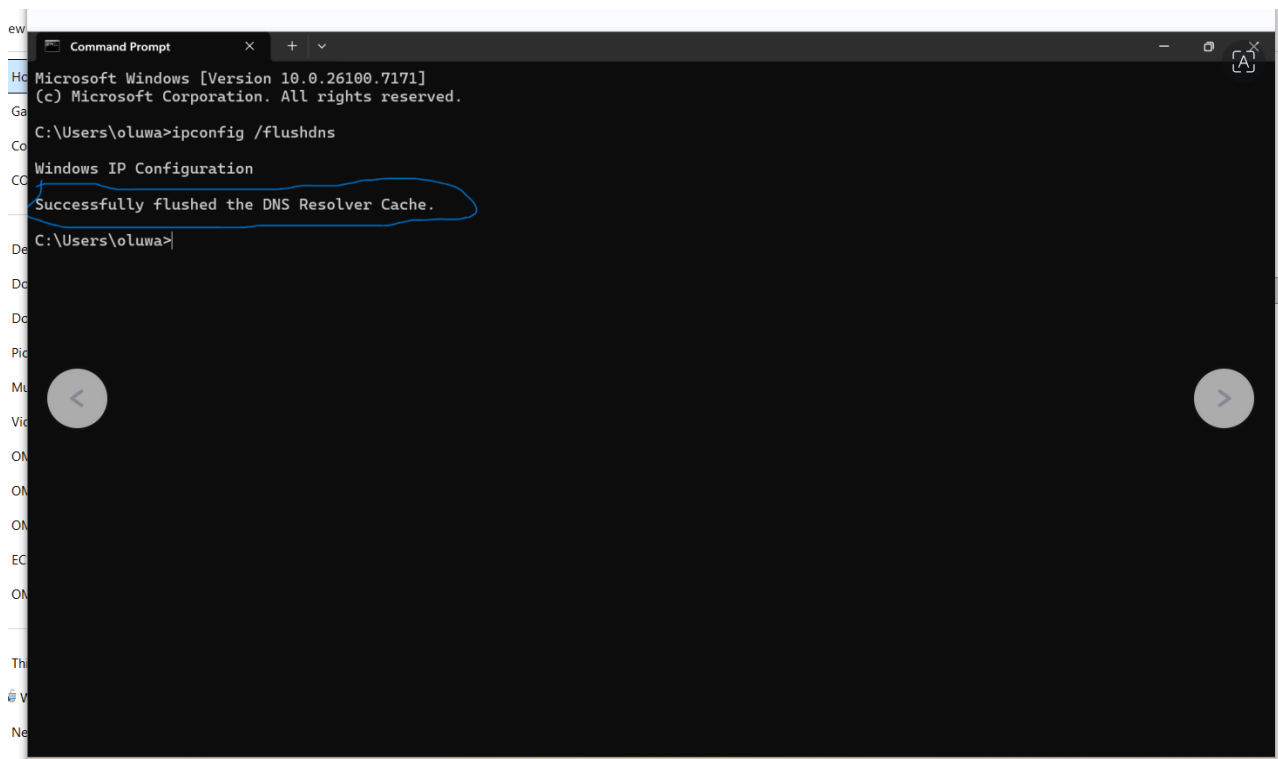
In this project, I performed a packet-level analysis of **Domain Name System (DNS)** behavior by combining command-line diagnostics (nslookup) with network traffic inspection in **Wireshark**. The objective was to understand how DNS queries are generated, how responses are returned, how caching affects name resolution, and how DNS resolution enables HTTP and HTTPS communication.

This investigation was conducted from a **defensive and forensic perspective**, mirroring how SOC analysts analyze DNS traffic during incident response, threat hunting, and network troubleshooting.

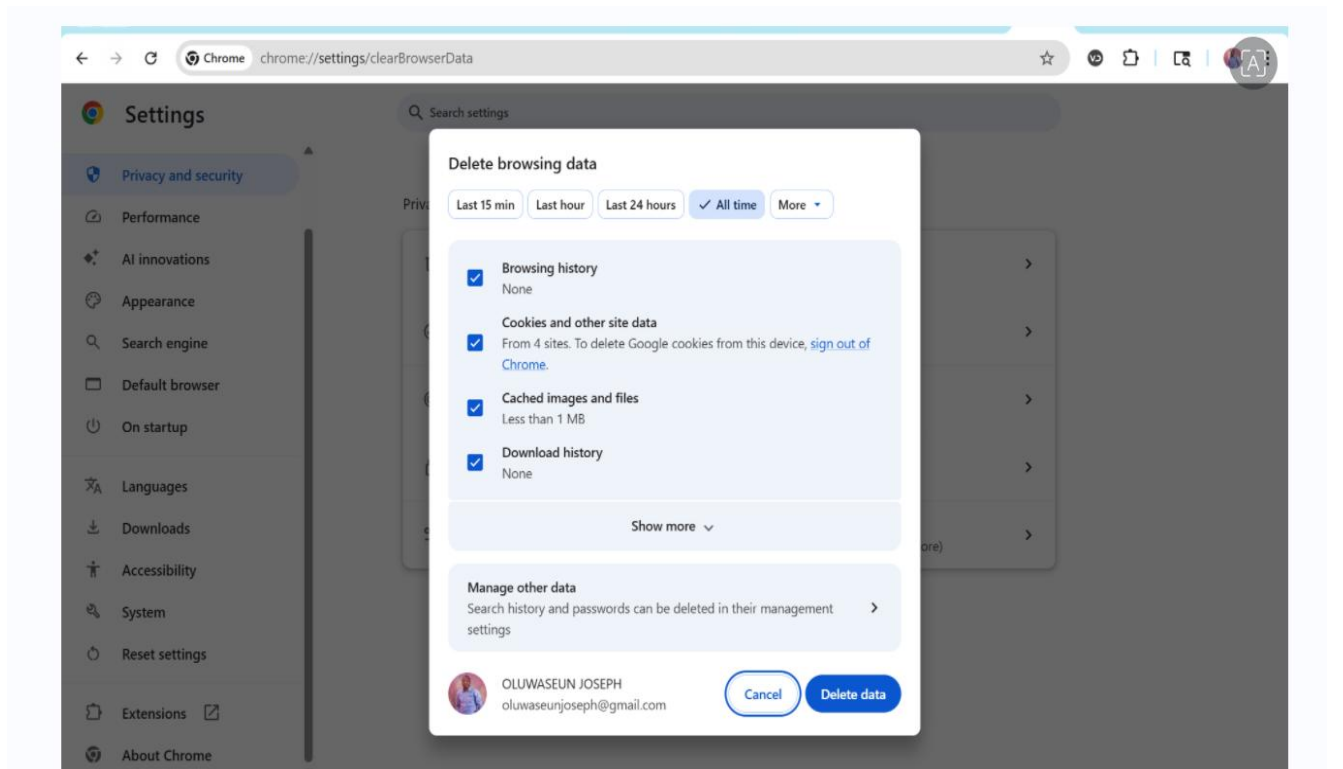
## 2. Environment & Preparation

- **Operating System:** Windows 10
- **Packet Analysis Tool:** Wireshark
- **Command-line Tool:** nslookup
- **Observed Protocols:** DNS (UDP/53), HTTP, HTTPS

Before capturing traffic, I cleared both the **local DNS resolver cache** and **browser cache** to ensure that all DNS queries observed during the capture represented fresh network requests.



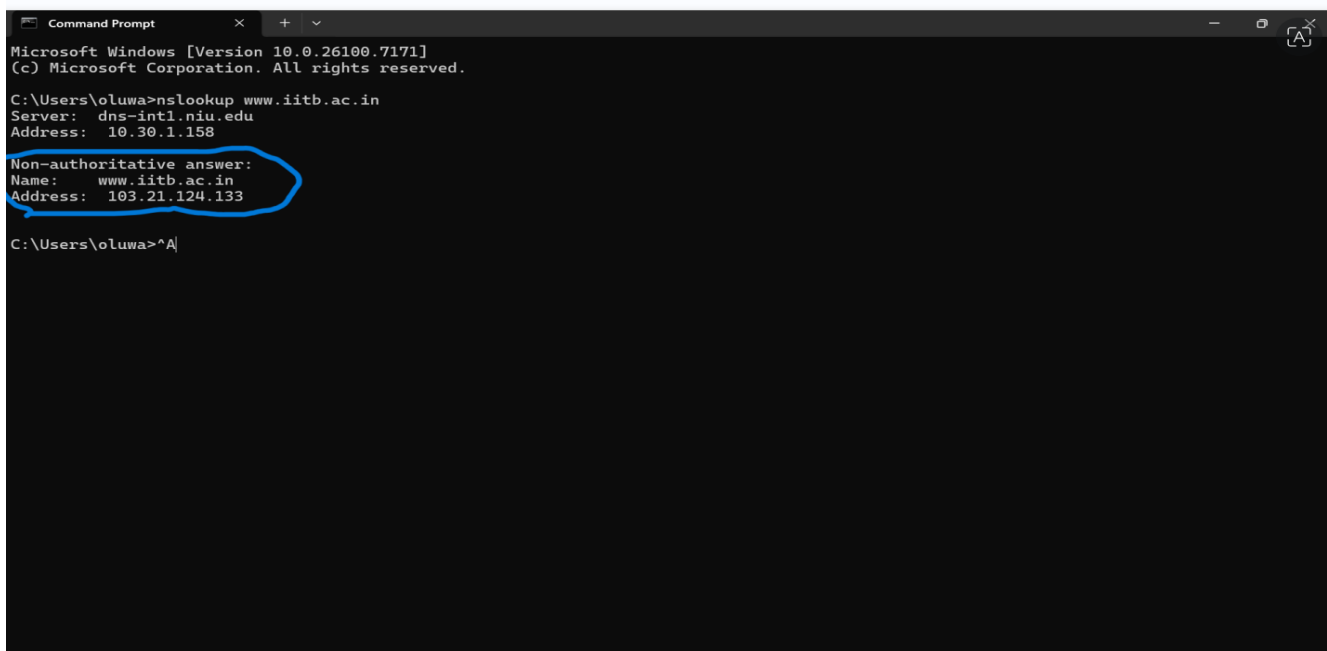
**Figure iv – Successfully clearing the local DNS cache**  
(Screenshot file: dns-cache-flush.png)



**Figure v – Browser cache cleared**  
( Screenshot file: browser-cache-clear.png)

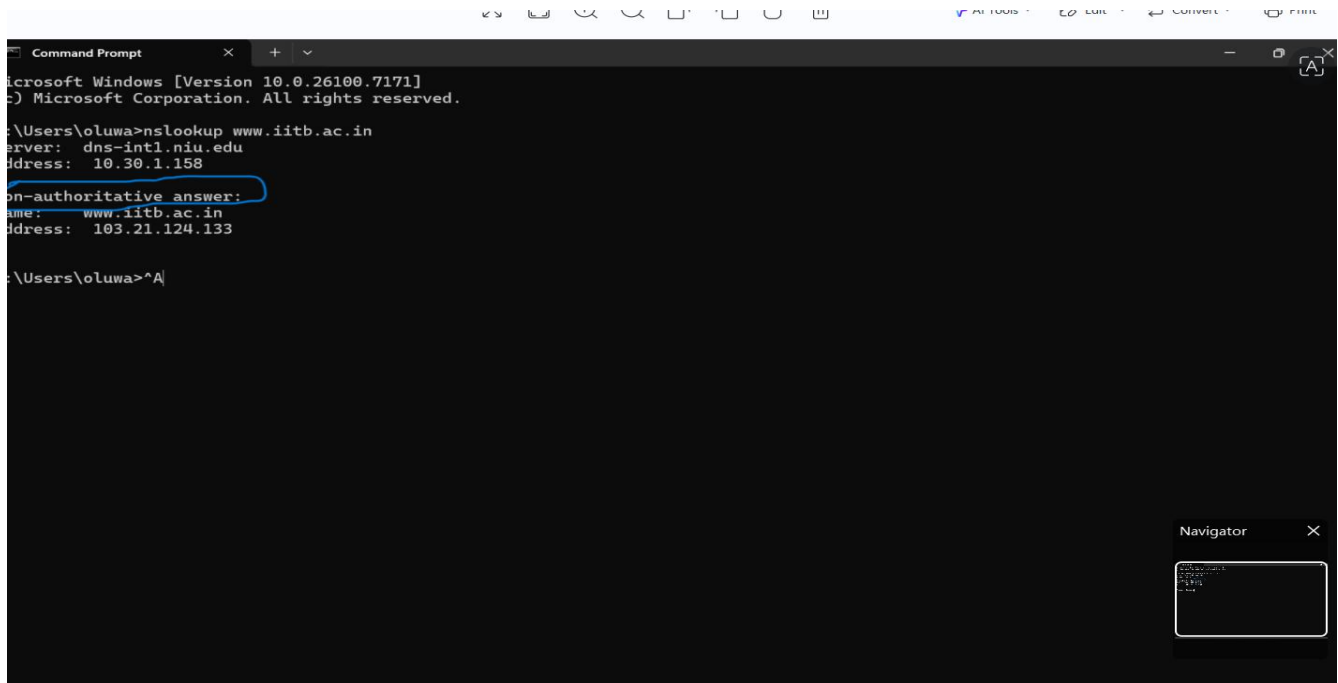
### 3. DNS Enumeration Using nslookup

To establish baseline DNS behavior, I used nslookup to perform record enumeration.



**Figure i – Basic nslookup command output**  
( Screenshot file: dns-nslookup-basic.png)

The response indicated that the result was non-authoritative, meaning it was returned by a recursive resolver rather than an authoritative name server.



```
Microsoft Windows [Version 10.0.26100.7171]
(c) Microsoft Corporation. All rights reserved.

C:\Users\oluwa>nslookup www.iitb.ac.in
Server: dns-int1.niu.edu
Address: 10.30.1.158

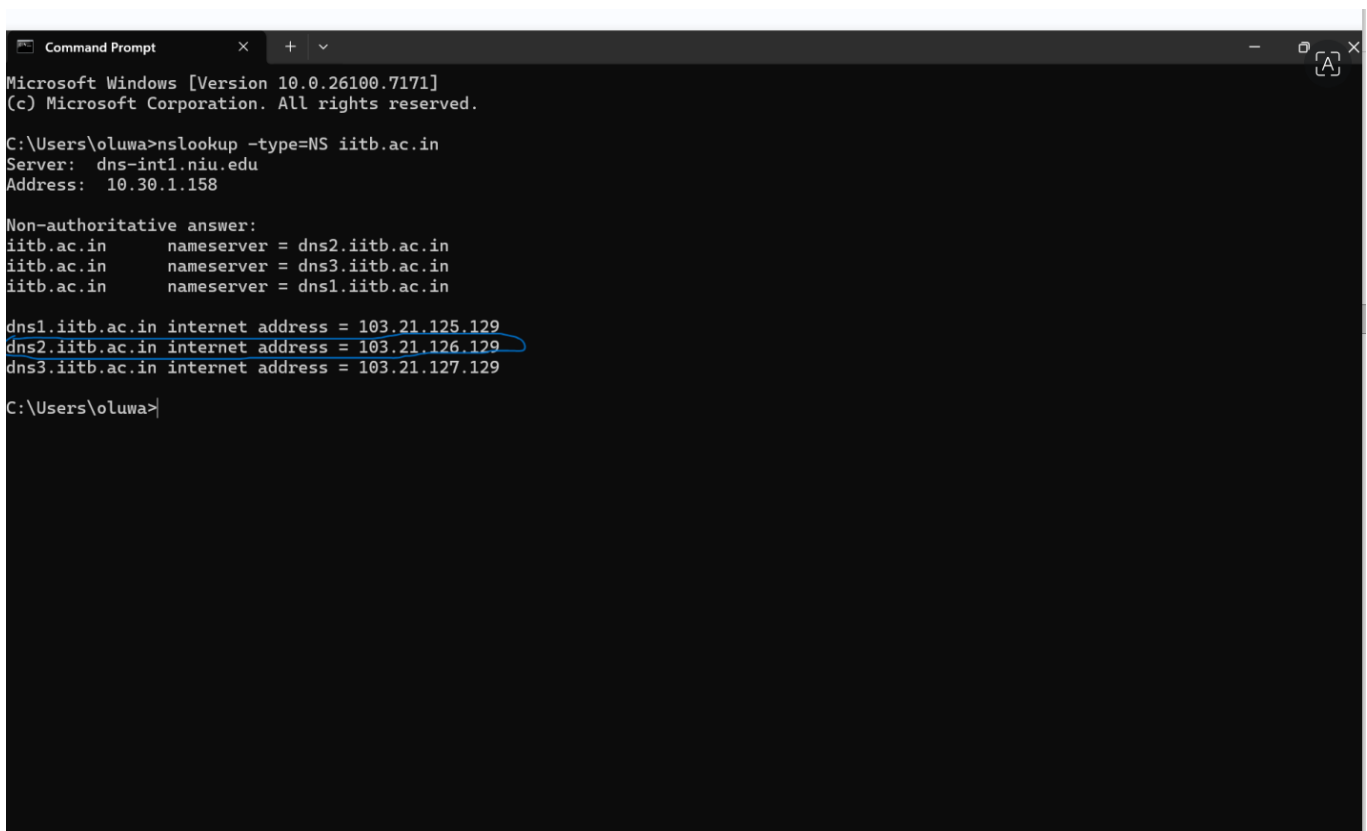
Non-authoritative answer:
Name: www.iitb.ac.in
Address: 103.21.124.133

C:\Users\oluwa>^A
```

**Figure ii – Non-authoritative DNS response**

(Screenshot file: dns-non-authoritative.png)

I then queried **NS records** to identify authoritative name servers for the domain.



```
Microsoft Windows [Version 10.0.26100.7171]
(c) Microsoft Corporation. All rights reserved.

C:\Users\oluwa>nslookup -type=NS iitb.ac.in
Server: dns-int1.niu.edu
Address: 10.30.1.158

Non-authoritative answer:
iitb.ac.in      nameserver = dns2.iitb.ac.in
iitb.ac.in      nameserver = dns3.iitb.ac.in
iitb.ac.in      nameserver = dns1.iitb.ac.in

dns1.iitb.ac.in internet address = 103.21.125.129
dns2.iitb.ac.in internet address = 103.21.126.129
dns3.iitb.ac.in internet address = 103.21.127.129

C:\Users\oluwa>
```

**Figure iii – NS record query showing authoritative name servers**

(Screenshot file: dns-ns-authoritative-records.png)

## 4. Wireshark DNS Capture Initialization

After clearing caches, I launched Wireshark and initiated a packet capture to record DNS and HTTP traffic generated during web access and command-line lookups.

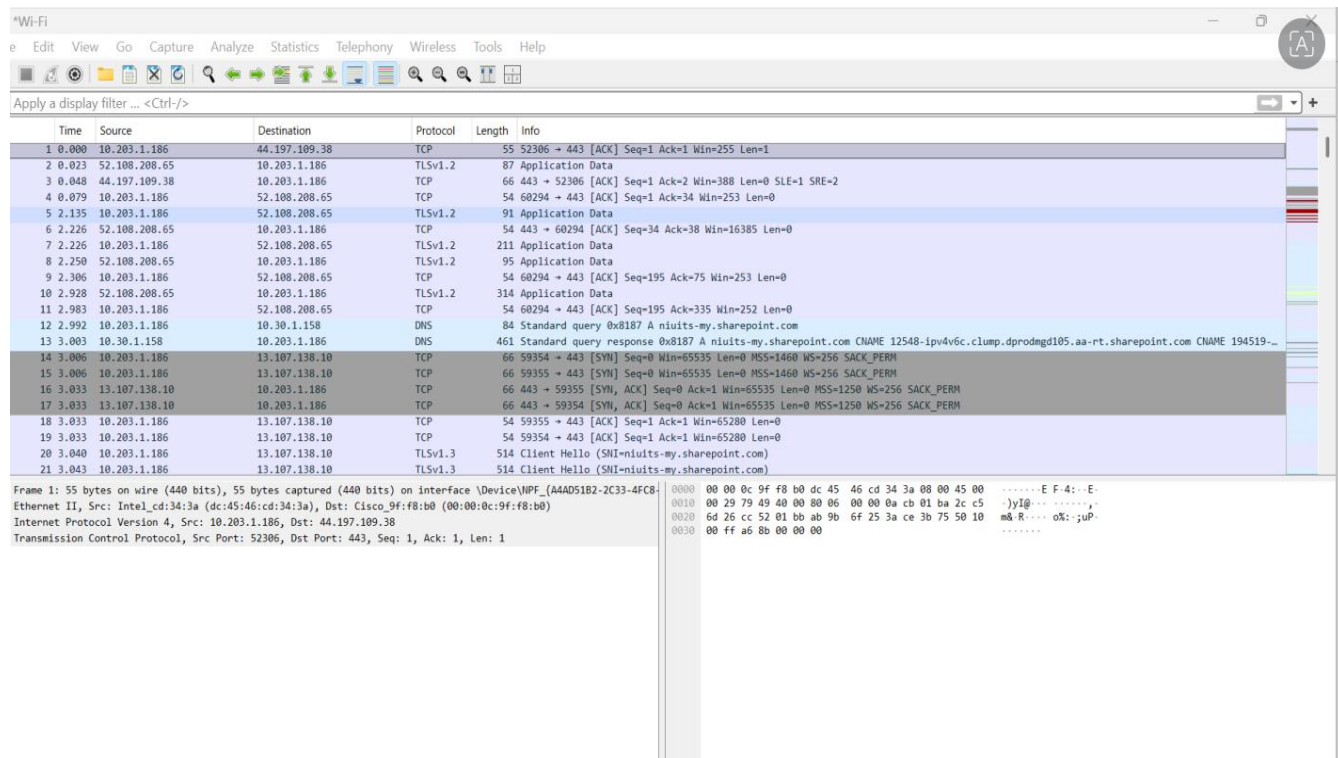


Figure vi – Wireshark capture stopped after traffic generation  
(Screenshot file: wireshark-capture-stopped.png)

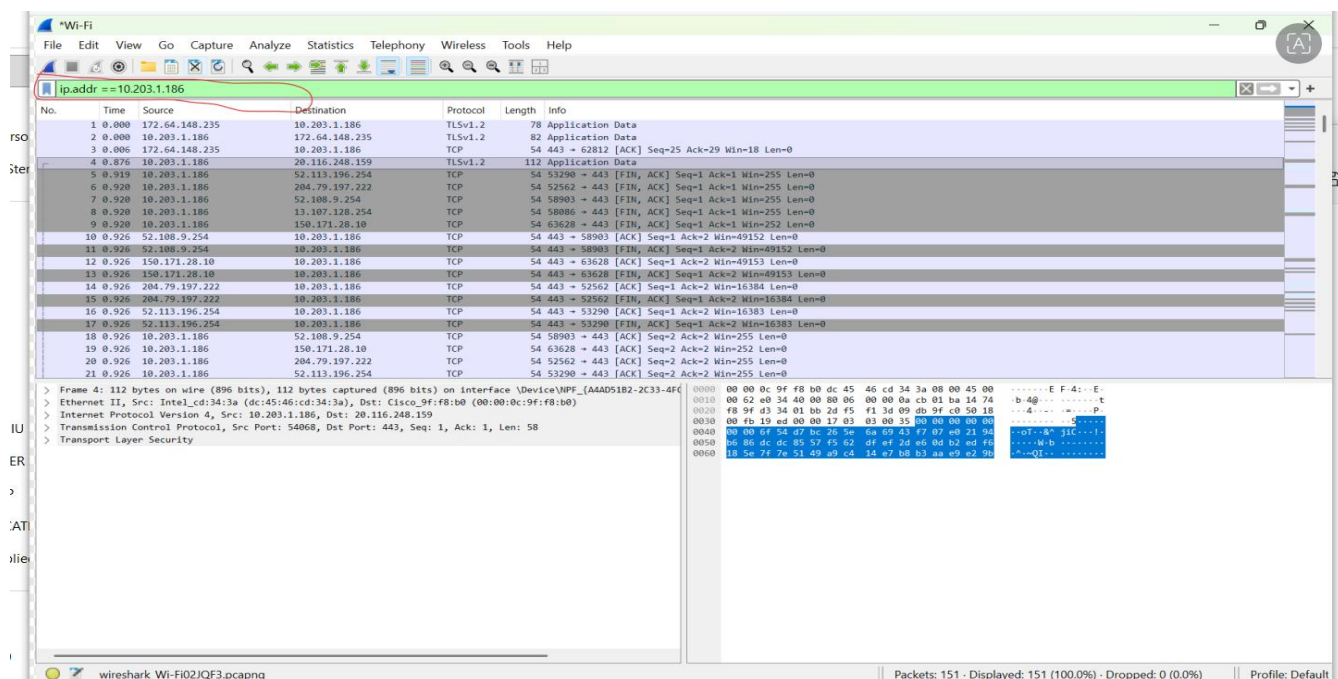
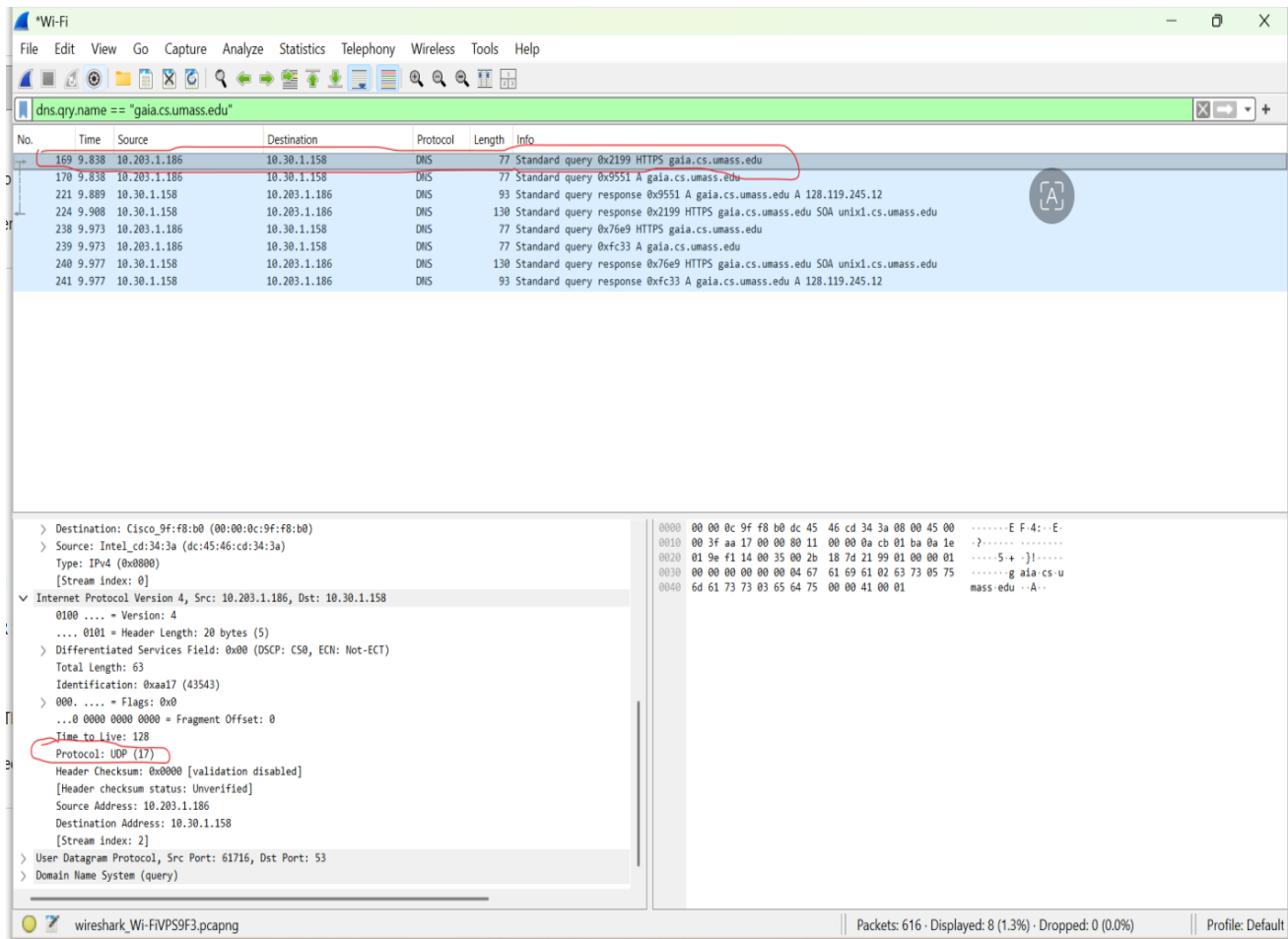


Figure vii – Tracing DNS packets in Wireshark  
(Screenshot file: wireshark-dns-packet-trace.png)

## 5. DNS Query and Response Analysis

### 5.1 Initial DNS Query

The first DNS query attempting to resolve `gaia.cs.umass.edu` appeared in **Frame 169**. Inspection of the packet showed that the query was sent using **UDP** with destination port **53**, the standard DNS server port.



**Figure 5 – Initial DNS query for `gaia.cs.umass.edu` (Frame 169)**  
(Screenshot file: `dns-query-initial.png`)

## 5.2 DNS Response

The corresponding DNS response was identified in **Frame 224**, as indicated by the “Response In” field in the DNS header of Frame 169. The response was also transmitted over UDP.

The screenshot displays a Wireshark packet capture of a network traffic analysis. The top pane shows a list of packets with the following details:

| No. | Time  | Source       | Destination  | Protocol | Length | Info                                                                          |
|-----|-------|--------------|--------------|----------|--------|-------------------------------------------------------------------------------|
| 169 | 9.838 | 10.203.1.186 | 10.30.1.158  | DNS      | 77     | Standard query 0x2199 HTTPS gaia.cs.umass.edu                                 |
| 170 | 9.838 | 10.203.1.186 | 10.30.1.158  | DNS      | 77     | Standard query 0x9551 A gaia.cs.umass.edu                                     |
| 221 | 9.889 | 10.30.1.158  | 10.203.1.186 | DNS      | 93     | Standard query response 0x9551 A gaia.cs.umass.edu A 128.119.245.12           |
| 224 | 9.908 | 10.30.1.158  | 10.203.1.186 | DNS      | 130    | Standard query response 0x2199 HTTPS gaia.cs.umass.edu SOA unix1.cs.umass.edu |
| 238 | 9.973 | 10.203.1.186 | 10.30.1.158  | DNS      | 77     | Standard query 0x76e9 HTTPS gaia.cs.umass.edu                                 |
| 239 | 9.973 | 10.203.1.186 | 10.30.1.158  | DNS      | 77     | Standard query 0xfc33 A gaia.cs.umass.edu                                     |
| 240 | 9.977 | 10.30.1.158  | 10.203.1.186 | DNS      | 130    | Standard query response 0x76e9 HTTPS gaia.cs.umass.edu SOA unix1.cs.umass.edu |
| 241 | 9.977 | 10.30.1.158  | 10.203.1.186 | DNS      | 93     | Standard query response 0xfc33 A gaia.cs.umass.edu A 128.119.245.12           |

The bottom pane shows the details of the selected packet (Frame 224), which is a DNS response. The details are as follows:

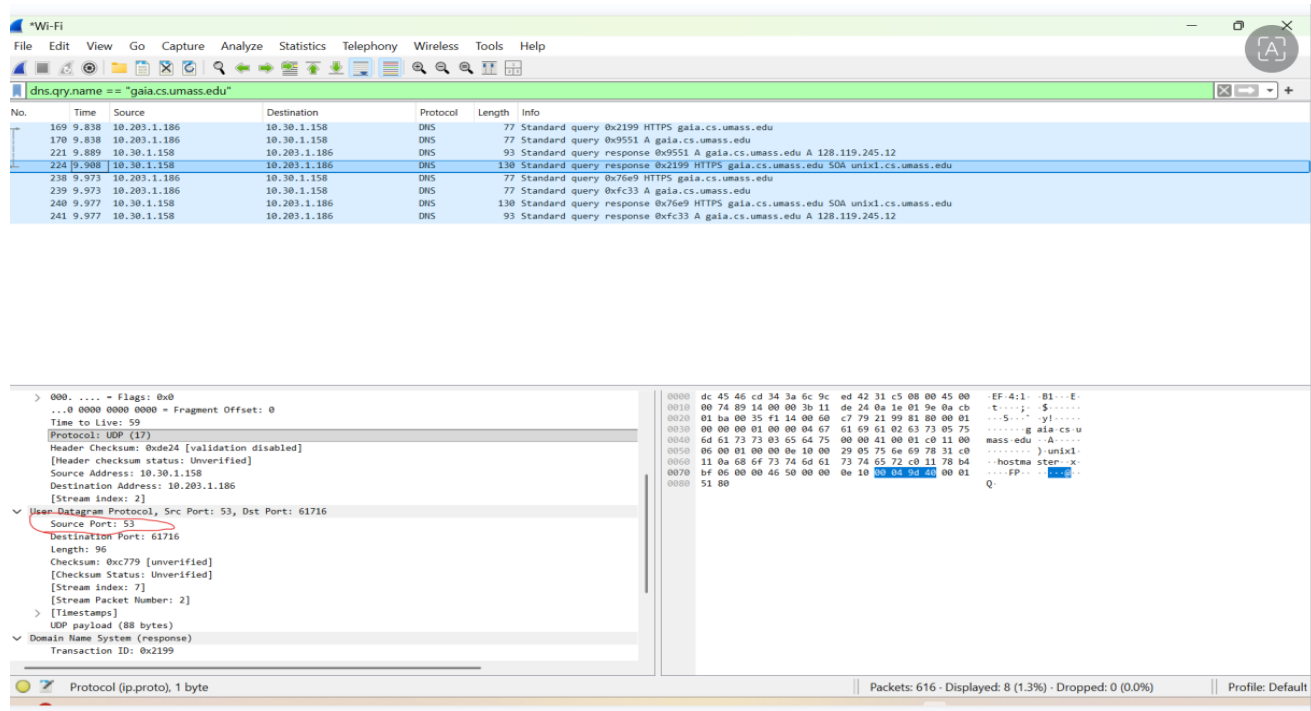
- Identification: 0x8914 (35092)
- 0000 ..... = Flags: 0x0
- ...0 0000 0000 0000 = Fragment Offset: 0
- Time to live: 59
- Protocol: UDP (17)
- Header checksum: 0xde24 [validation disabled]
- [Header checksum status: Unverified]
- Source Address: 10.30.1.158
- Destination Address: 10.203.1.186
- [Stream index: 2]
- User Datagram Protocol, Src Port: 53, Dst Port: 61716
- Domain Name System (response)
- Transaction ID: 0x2199
- Flags: 0x8180 Standard query response, No error
- Questions: 1
- Answer RRs: 0
- Authority RRs: 1
- Additional RRs: 0
- Queries
- Authoritative nameservers
- [Request In: 169]

The packet data is displayed in hexadecimal and ASCII format on the right side of the details pane.

**Figure 6 – DNS response corresponding to Frame 169 (Frame 224)**  
(Screenshot file: dns-response-ip-resolution.png)

## 5.3 DNS Port Behavior

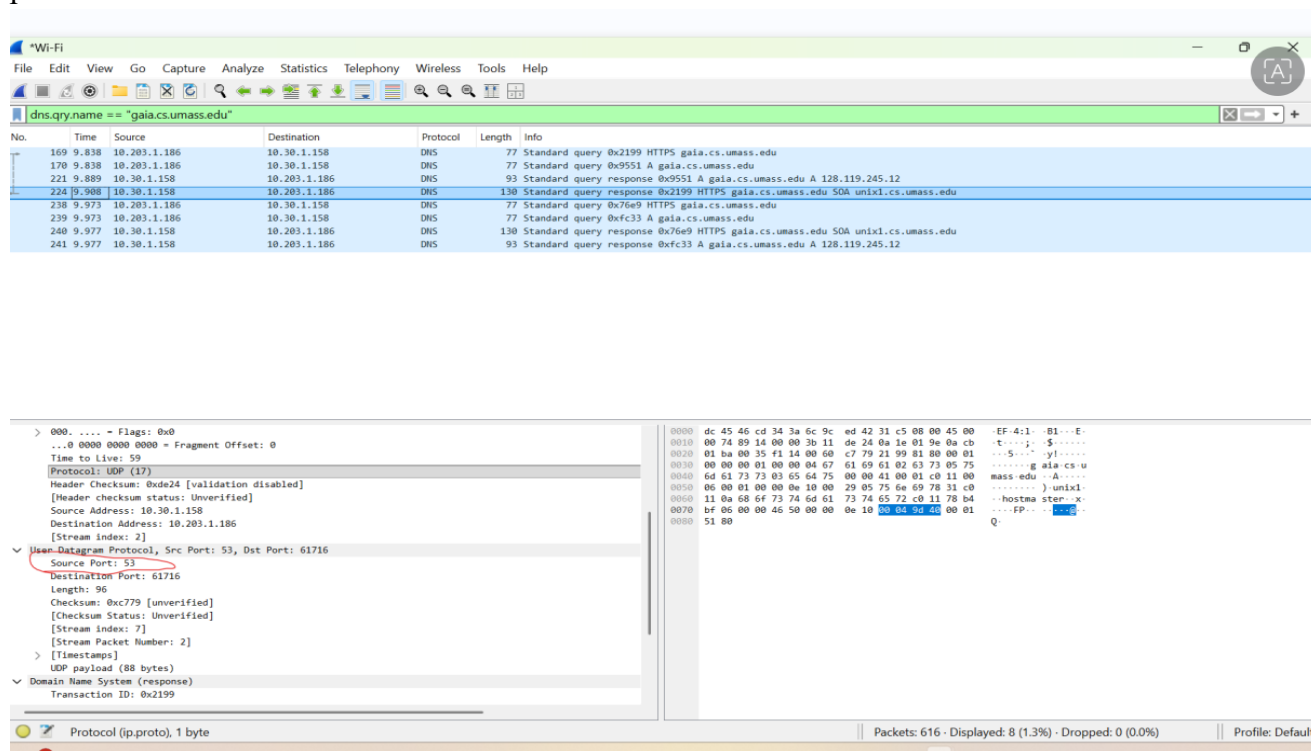
The DNS query originated from an **ephemeral client port (61716)** and was sent to destination port **53**.



**Figure 7a – Destination port of DNS query message**

(Screenshot file: dns-dest-port.png)

The response reversed the port pairing, originating from port **53** and returning to the client's ephemeral port.



**Figure 7b – Source port of DNS response message**

(Screenshot file: dns-source-port.png)



## 5.4 DNS Server Identification

The DNS query was sent to the resolver at IP address **10.30.1.158**, representing the DNS server handling the request.

The screenshot shows a Wireshark packet capture on the 'Wi-Fi' interface. The filter bar at the top is set to 'dns.qry.name == "gaia.cs.umass.edu"'. The packet list pane shows several DNS packets. The first packet (No. 169) is a standard query to 10.30.1.158 for 'gaia.cs.umass.edu'. The packet details pane for this packet shows the following information:

- Protocol: UDP (17)
- Header Checksum: 0x0000 [validation disabled]
- [Header checksum status: Unverified]
- Source Address: 10.203.1.186
- Destination Address: 10.30.1.158
- [Stream index: 2]
- User Datagram Protocol, Src Port: 61716, Dst Port: 53
- Source Port: 61716
- Destination Port: 53
- Length: 43
- Checksum: 0x187d [unverified]
- [Checksum Status: Unverified]
- [Stream index: 7]
- [Stream Packet Number: 1]
- [Timestamps]
- UDP payload (35 bytes)
- Domain Name System (query)
- Transaction ID: 0x2199

The packet bytes pane shows the raw data of the packet, including the Ethernet II header, Internet Protocol Version 4 header, and User Datagram Protocol header.

 **Figure 8 – Destination IP address of DNS query**  
(Screenshot file: dns-destination-ip.png)



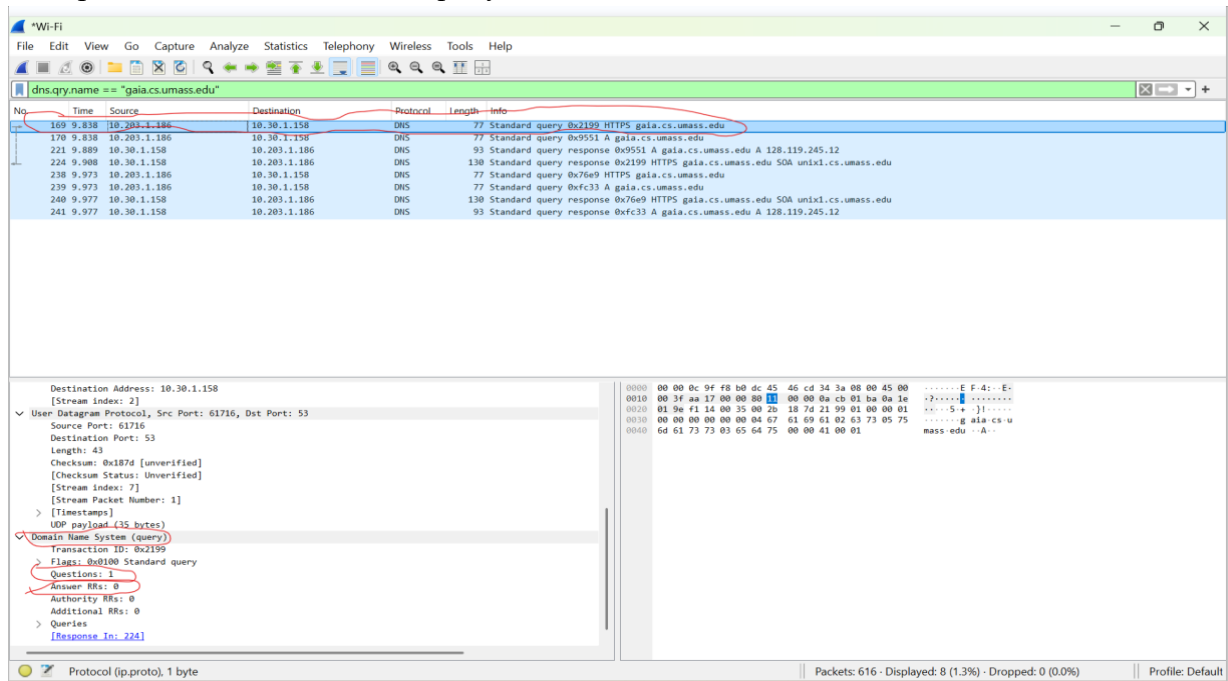
## 6. DNS Message Structure Analysis

### 6.1 DNS Query Header

The DNS query packet contained:

- 1 Question
- 0 Answers

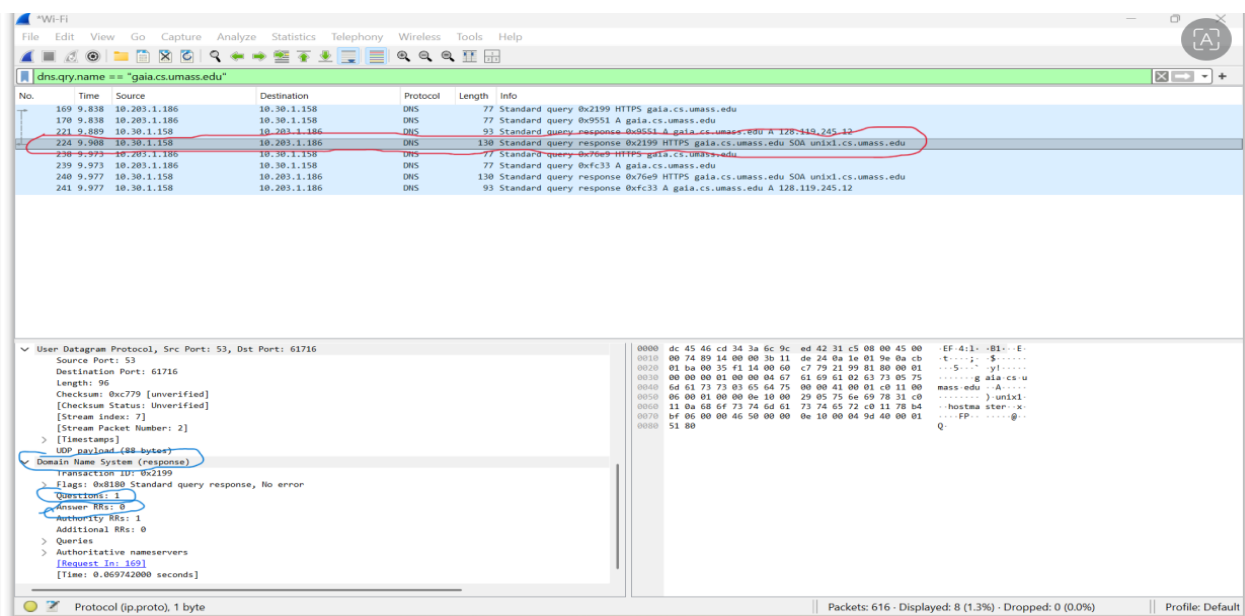
This is expected behavior for a DNS query.



**Figure 9 – DNS query header showing question and answer count**  
(Screenshot file: dns-query-header-fields.png)

### 6.2 DNS Response Header

The DNS response packet repeated the original question and included the resolved IP address.



**Figure 10 – DNS response header with question and answer fields**  
(Screenshot file: dns-response-header-fields.png)

## 7. DNS and HTTP Correlation

### 7.1 HTTP Request After DNS Resolution

After DNS resolution completed, the browser issued an HTTP GET request for the base resource.

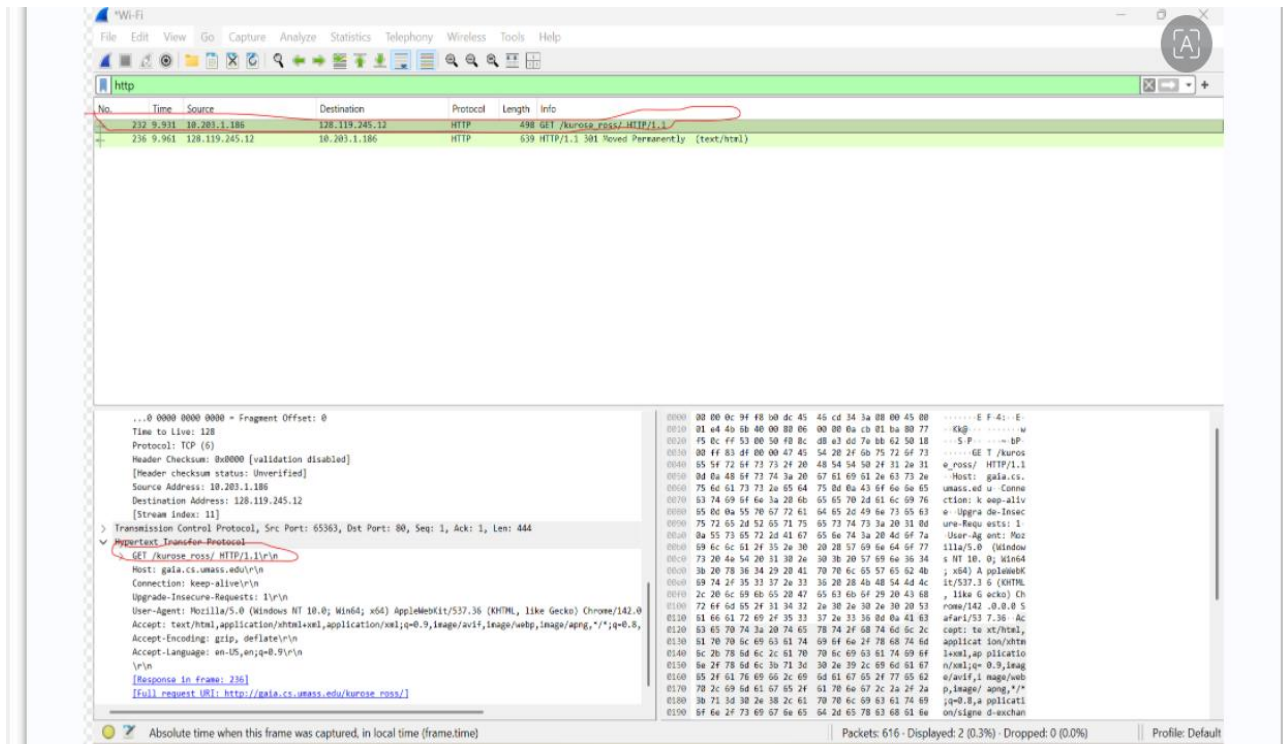


Figure 11a – Initial HTTP GET request (Frame 232)

(Screenshot file: http-get-after-dns.png)

### 7.2 DNS Resolution Supporting HTTP

The DNS A-record query for gaia.cs.umass.edu appeared in **Frame 170**, followed by the matching response in **Frame 221**, resolving the hostname to **128.119.245.12**.

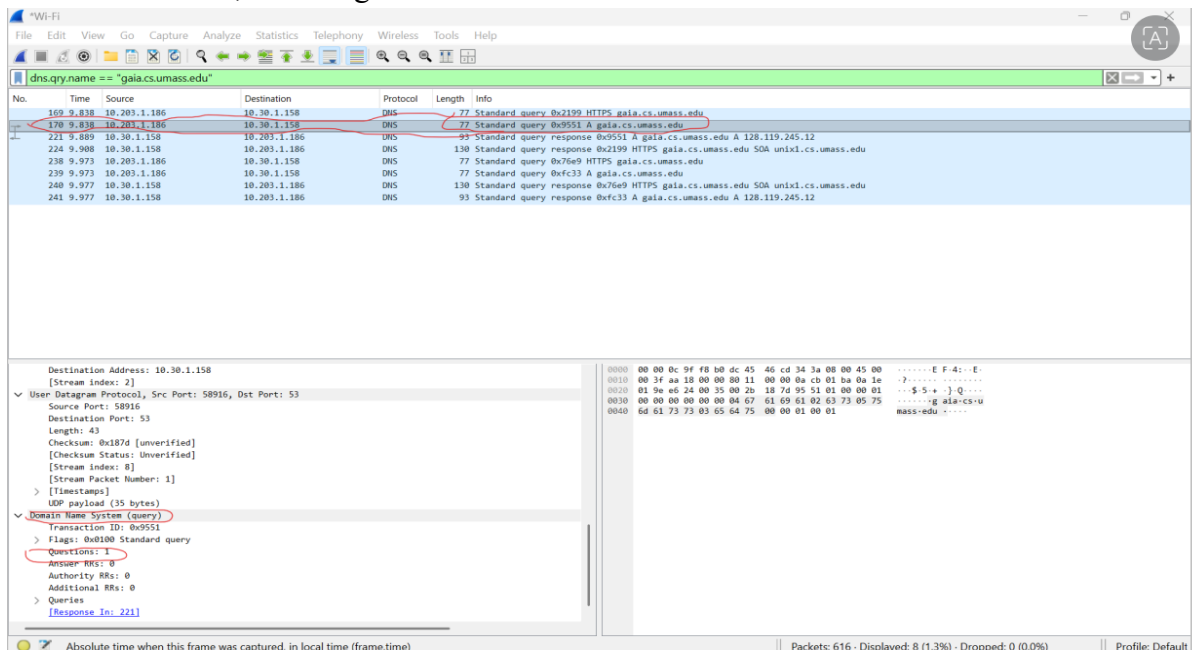


Figure 11b – DNS A-record query for gaia.cs.umass.edu

(Screenshot file: dns-a-record-query.png)

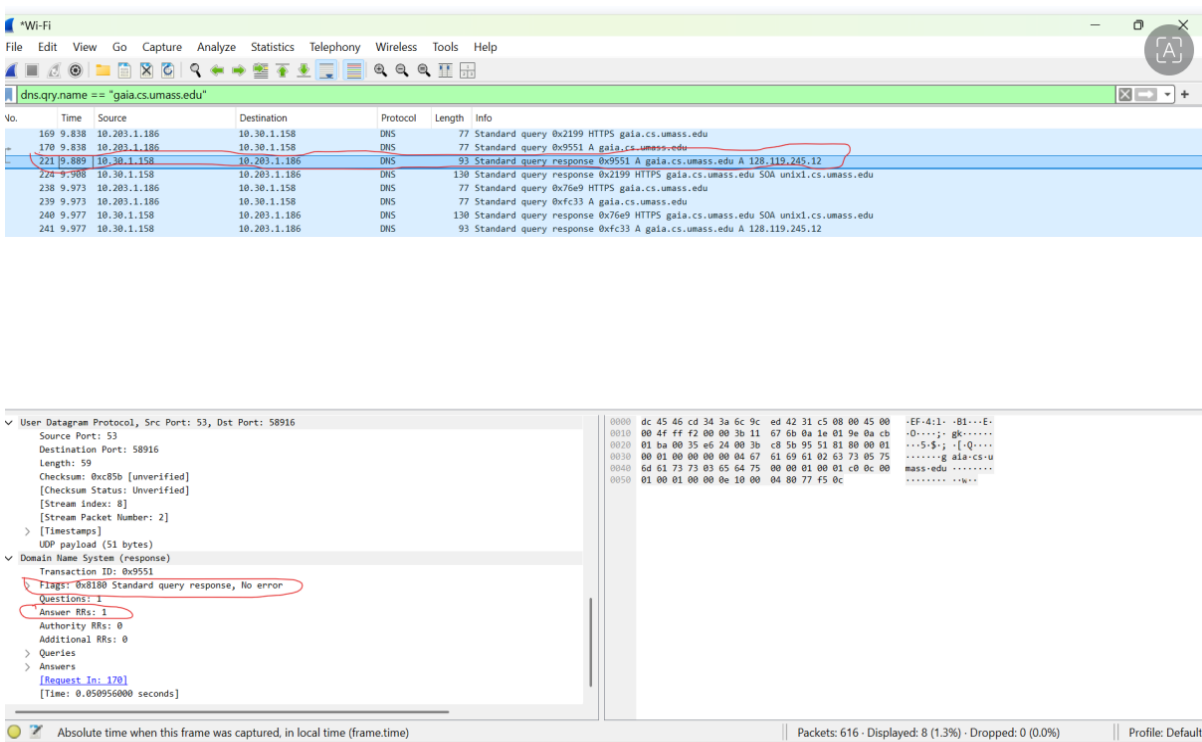


Figure 11c – DNS A-record response with resolved IP address

(Screenshot file: dns-a-record-response.png)

### 7.3 HTTPS Transition and Caching

After the initial HTTP request, the server issued a **301 redirect**, and subsequent traffic transitioned to HTTPS. Because the DNS result was cached, no additional DNS queries were generated for subsequent HTTPS requests.

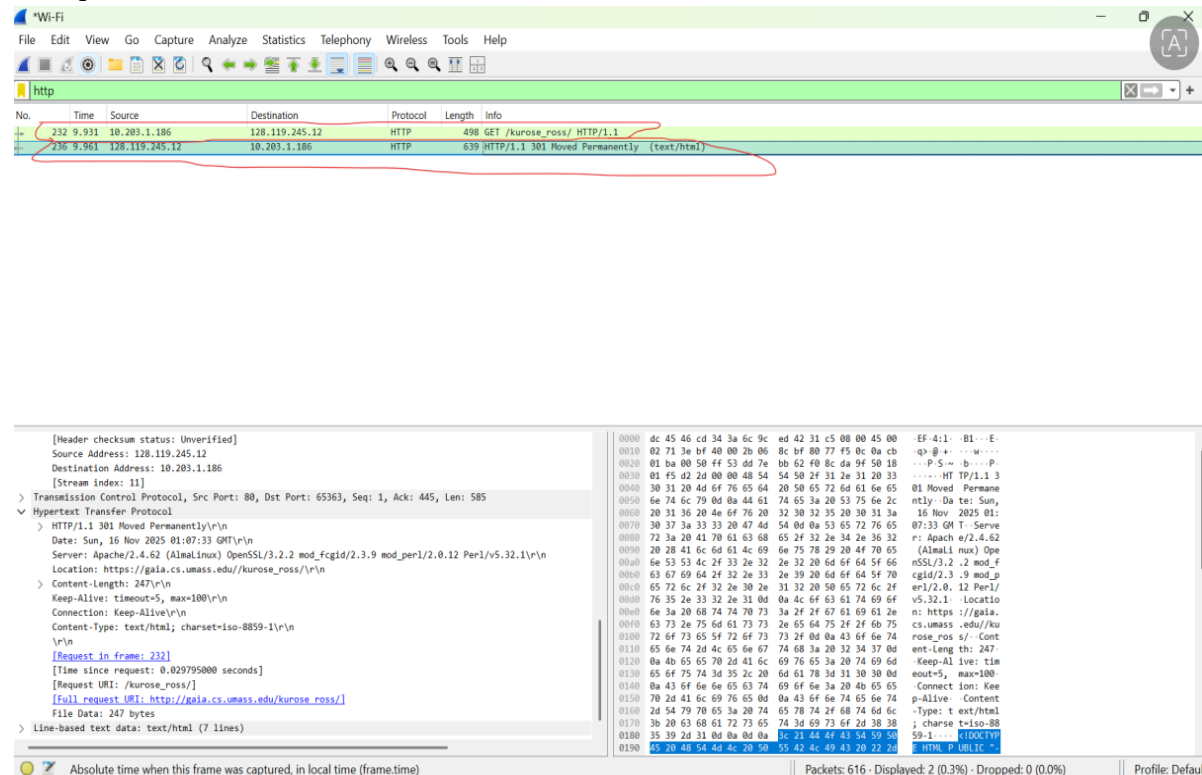
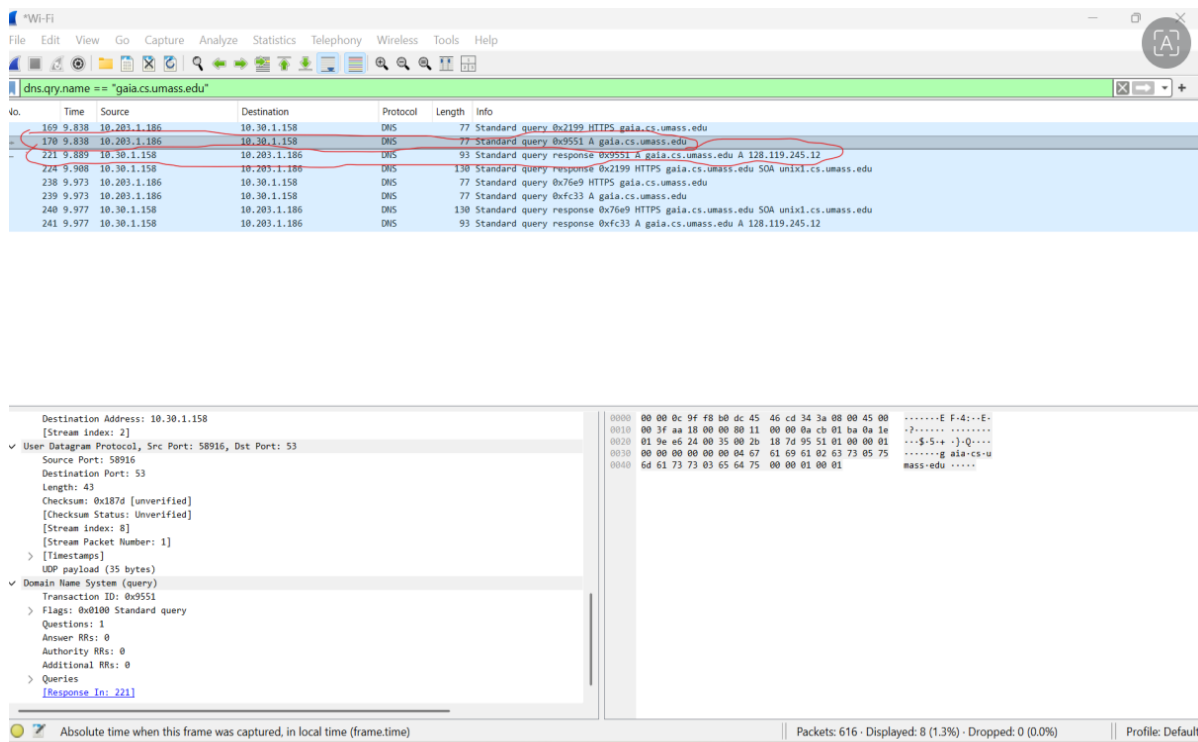


Figure 11d – HTTP 301 redirect to HTTPS

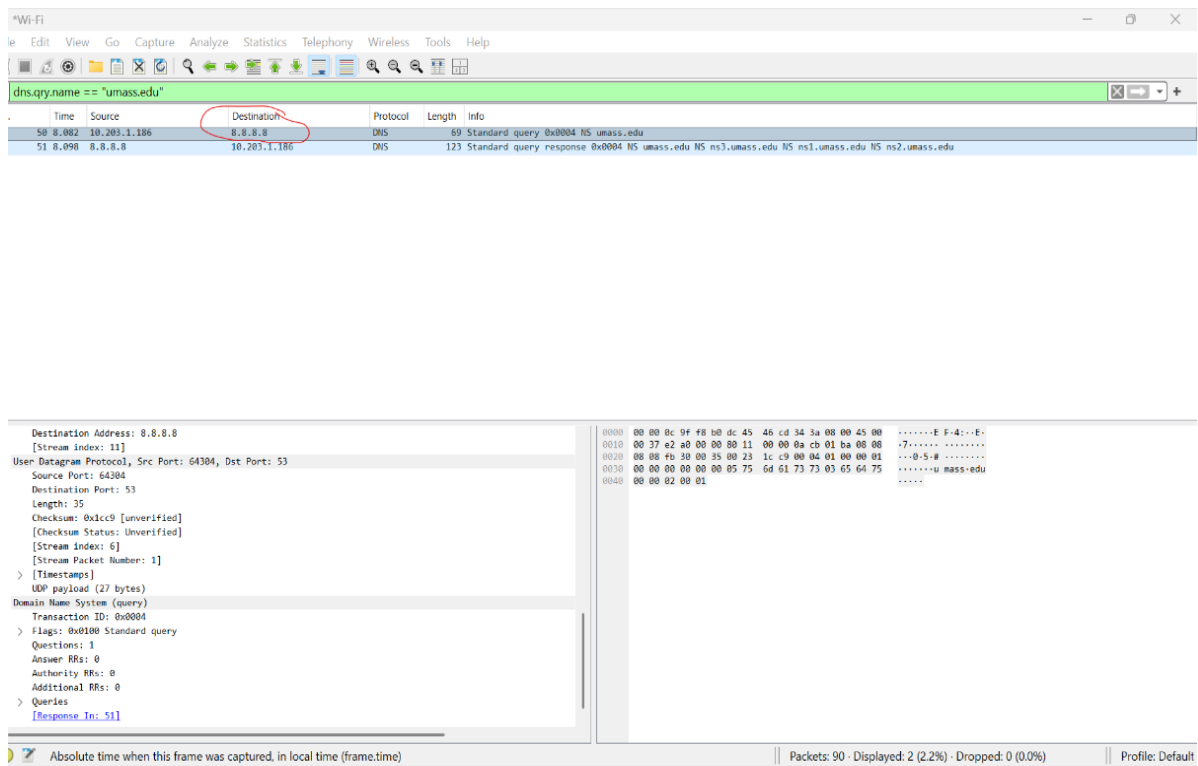
(Screenshot file: http-to-https-redirect.png)



**Figure 11e – Cached DNS result reused without new query**  
(Screenshot file: dns-cache-hit.png)

## 8. External DNS Resolver Validation

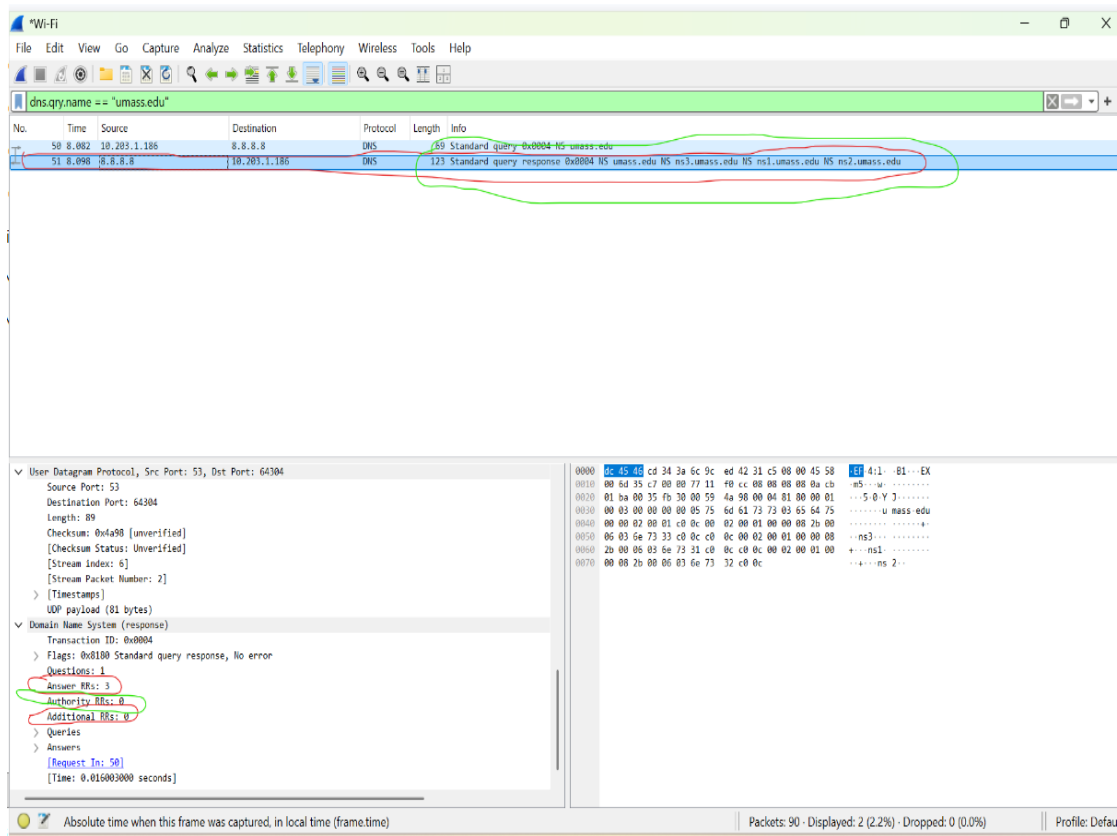
To bypass the local DNS cache, I performed DNS queries using **Google's public DNS server (8.8.8.8)**.



**Figure 16 – DNS query sent to Google DNS (8.8.8.8)**  
(Screenshot file: dns-google-resolver-query.png)

## 9. Authoritative Name Server Analysis

The DNS response for the NS query contained **three authoritative name servers** for umass.edu, confirming authoritative delegation.



**Figure 18 – DNS response showing authoritative NS records**  
(Screenshot file: dns-ns-authoritative-response.png)

## 10. Conclusion

This project demonstrated how DNS resolves hostnames to IP addresses and how caching significantly reduces redundant network queries. By combining nslookup diagnostics with Wireshark packet analysis, I observed real DNS query and response behavior, transport-layer port usage, message structure, and the dependency between DNS and HTTP/HTTPS traffic.

This investigation strengthened my ability to interpret DNS activity from a **SOC and forensic perspective**, supporting skills in **incident response, network troubleshooting, and threat analysis**.